# ONECLICKQA

# DETAILED TECHNICAL PRODUCT REQUIREMENTS & SYSTEM DESIGN

*AI-Native Autonomous QA Platform • Development Master Specification*

| Document control | Value |
|---|---|
| Document | OneClickQA Detailed Technical Product Requirements & System Design |
| Version | 2.0 — Development Baseline |
| Date | August 2026 |
| Audience | Product, QA Architect, Backend, Frontend, AI/ML, SDET, DevOps/SRE, Security, QA, Integration Engineers |
| Purpose | Define what must be built, how each module behaves, how users interact, how information is processed internally, how services communicate, and how the platform is validated. |
| Status | Development specification; implementation-level details are proposals unless explicitly identified as source requirements. |

This document is intentionally more detailed than a conventional PRD. It combines product requirements, functional requirements, technical design, module specifications, user stories, workflows, UI behavior, processing rules, data contracts, API contracts, integrations, testing architecture, defect management, security and engineering standards.

Source basis: the supplied OneClickQA Autonomous QA Product Blueprint and Complete Feature List/Feature Matrix, the supplied architecture/feature reference visuals, prior OneClickQA project discussions, and the technical-document structure used in the supplied reference documentation. The blueprint defines the seven-layer architecture and the closed-loop autonomous QA workflow; the feature matrix defines the complete testing and platform capability catalogue. filecite turn7file2 filecite turn9file15

# 1. How This Document Must Be Used

This is the master technical requirements document for engineering. A feature is not considered fully specified merely because its name appears in a feature list. Each implementation must answer: what the user sees, what input is accepted, what validation occurs, what internal services process it, what data is created/updated, what AI agent/tool is invoked, what execution engine is used, what evidence is collected, what happens on failure, what APIs/events are emitted, what permissions apply, and what acceptance criteria prove completion.

| Requirement classification | Meaning | Action |
|---|---|---|
| SOURCE | Explicitly present in the supplied blueprint/feature matrix/reference. | Must be retained. Scope can be assigned to MVP or later phase, but must not be silently removed. |
| ARCHITECTURE DIRECTION | Technology/layer/service direction shown by source/reference. | Default implementation direction. Change requires an Architecture Decision Record (ADR). |
| IMPLEMENTATION DETAIL | Concrete endpoint, table, screen field, validation, state machine or algorithm added in this document. | Engineering baseline. Change requires review and traceability. |
| OPTIONAL/EXTENSION | Capability designed into the architecture but not required for the first release. | Do not block core architecture; implement through adapters/interfaces. |

The source blueprint's MVP is explicitly web-first: Chrome/Firefox/Edge, Playwright/Selenium, requirement-to-test generation, TestIntent™, test-data generation, one-click execution, evidence, AI failure analysis, basic confidence-based healing, core reporting and basic CI/CD. filecite turn8file14

# 2. Product Definition

OneClickQA accepts an application URL, API specification, business requirement, user story, existing automation, test assets or natural-language QA objective and turns that input into an evidence-backed QA lifecycle: understand → design → automate → execute → observe → diagnose → heal → prove → decide → learn. This is the central product promise in the source blueprint. filecite turn7file5

# 2.1 Product transformation

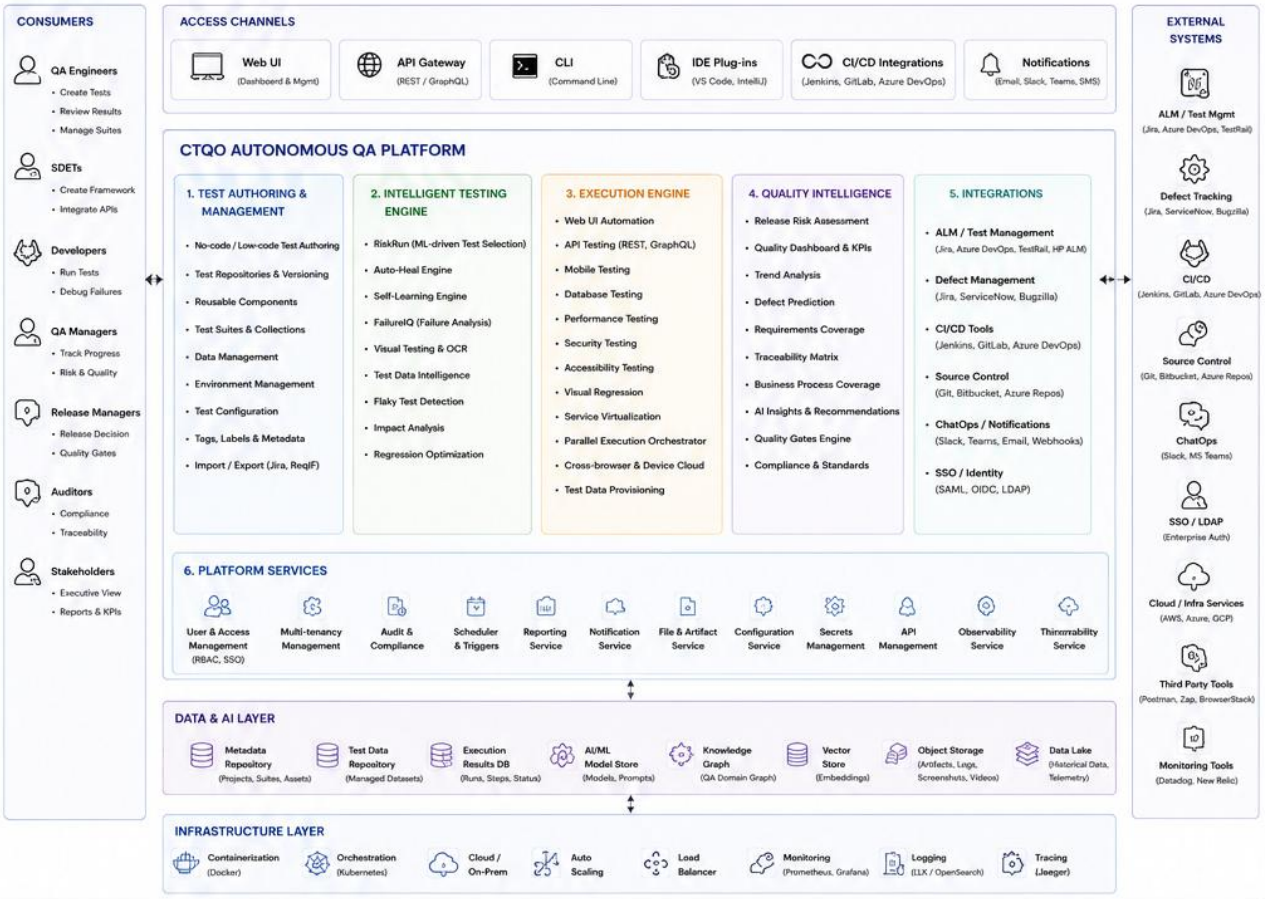
| Conventional QA | OneClickQA target |
|---|---|
| Write tests | Understand requirements and application |
| Manually identify objects | Discover and maintain AppGraph™ |
| Write automation first | Create TestIntent™ first |
| Run all regression tests | Use RiskRun™ to select/optimize regression |
| Inspect logs manually | FailureIQ™ aggregates and diagnoses evidence |
| Repair selectors manually | AutoHeal™ proposes/applies safe repairs |
| Collect screenshots separately | EvidenceGraph™ automatically links evidence |
| Create defect manually | AI Defect Intelligence prepares a defect package |
| Decide release using intuition | Release Intelligence produces explainable risk |
| Historical results remain fragmented | QA Knowledge Graph learns from history |

# 3. Reference Architecture — Required System Shape

The supplied architecture reference shows consumers/access channels, five primary product capability groups, platform services, a data/AI layer and infrastructure. The detailed lower-level reference decomposes the platform into API gateway, microservices, common services, execution framework, adapters, isolated workers, artifact/result services, data stores and infrastructure. The architecture below follows that direction.

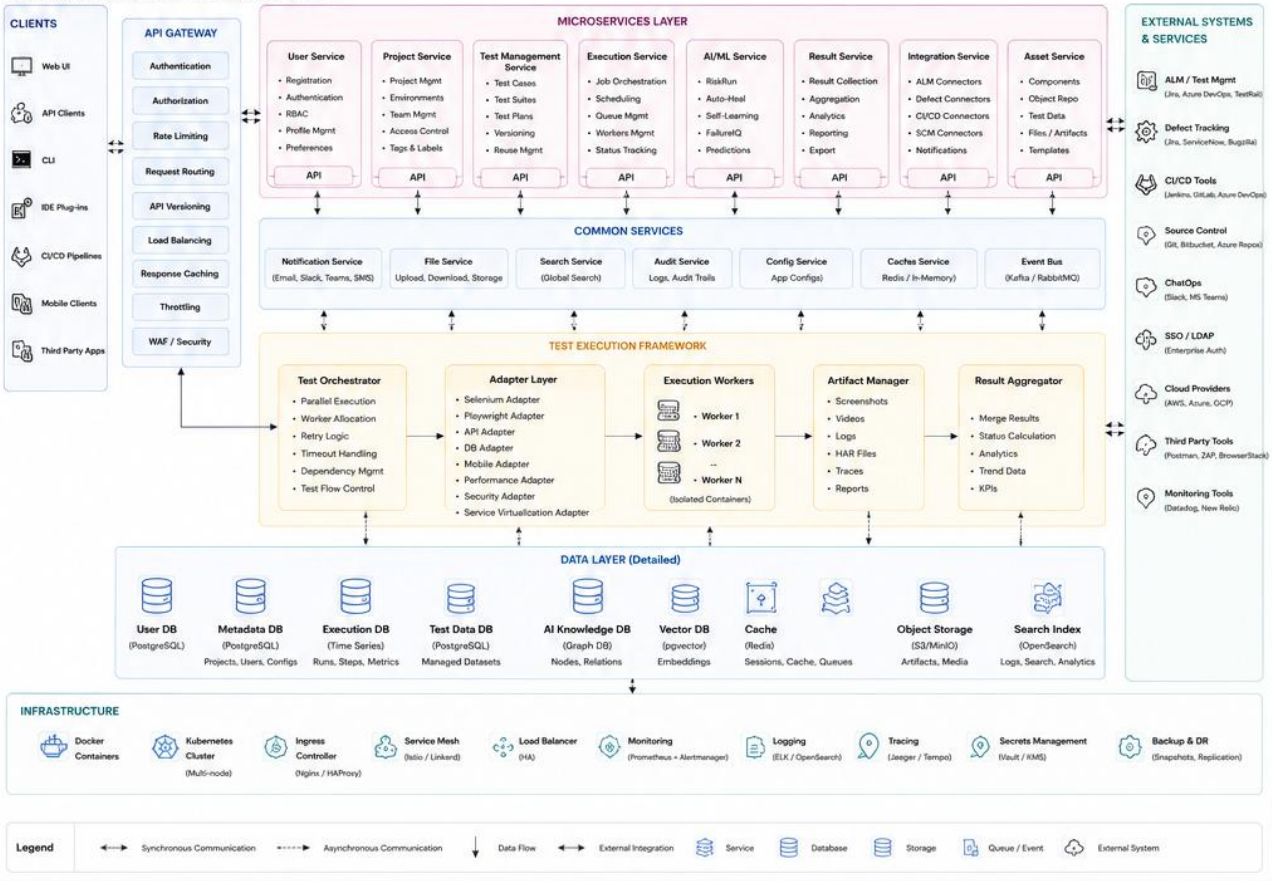
# HIGH LEVEL APPLICATION ARCHITECTURE

**CTQO Autonomous QA Platform** – High Level Architecture




# LOW LEVEL APPLICATION ARCHITECTURE

**CTQO Autonomous QA Platform** – Low Level Architecture



Reference architecture supplied with the project: high-level and low-level architecture, including access channels, five capability groups, platform services, data/AI layer, API gateway, microservices, common services, execution framework, adapters, workers, artifacts, result aggregation, data stores and infrastructure.

## 3.1 Seven logical product layers

| Layer | Responsibilities |
|---|---|
| L1 Experience | Web UI, command center, One-Click Test, Copilot, test explorer, evidence explorer, reports, administration |
| L2 AI Orchestration | Agent manager, workflow/state manager, policy engine, confidence engine, tool gateway |
| L3 Intelligence | Requirement, application, test, data, failure, healing, regression, defect intelligence |
| L4 Knowledge | AppGraph™, object repository, TestIntent™, requirements, defects, evidence graph, history, QA knowledge |
| L5 Automation | Playwright, Selenium, API, database, mobile, performance, security, accessibility and mock adapters |
| L6 Execution Fabric | Queue, scheduler, workers, browser/device farms, parallelization, isolation, retries, resource allocation |
| L7 Data & Infrastructure | PostgreSQL, pgvector, Redis, object storage, event bus, observability, secrets, deployment |

## 3.2 Logical service decomposition

| Service | Responsibilities |
|---|---|
| Identity & Tenant Service | Authentication, SSO, MFA, users, roles, permissions, tenant/project isolation |
| Project & Test Management Service | Projects, environments, requirements, test cases, suites, plans, versions, traceability |
| AI Orchestration Service | Agent execution, workflow state, tool calls, policy, confidence, approvals |
| Application Intelligence Service | Crawler, DOM/accessibility/network analysis, AppGraph™, application versions |
| Test Intelligence Service | Test generation, TestIntent™, test data, regression/risk, flakiness |
| Execution Service | Job orchestration, scheduling, worker allocation, adapters, retries, status |
| Evidence & Result Service | Artifacts, result aggregation, EvidenceGraph™, reports, dashboards |
| Integration Service | ALM, defect systems, source control, CI/CD, chatops, webhooks |
| Common Platform Services | Notification, file/artifact, search, configuration, cache, scheduler, audit, secrets, observability |

The source blueprint recommends starting with approximately eight logical services rather than prematurely creating dozens of microservices. filecite turn9file8

# 4. Users, Personas and Permission Model

| Persona | Primary needs | Typical actions |
|---|---|---|
| Admin | Control platform and tenant | Users, SSO, policies, integrations, secrets, quotas |
| QA Manager | Control quality | Plans, suites, approvals, risk, release, reports |
| QA Engineer | Design and validate quality | Requirements, tests, execution, evidence, defects |
| SDET/Automation Engineer | Build and maintain automation | Adapters, locators, automation artifacts, workers |
| Developer | Investigate failures quickly | Run tests, inspect evidence, debug defects, quality gate |
| Release Manager | Make release decisions | Risk, quality gates, approvals, release readiness |

| | | |
|---|---|---|
| Auditor | Prove compliance | Traceability, evidence, approvals, audit history |
| Executive/Stakeholder | See business quality | Risk, trend, ROI, release status |
| Viewer | Consume approved information | Dashboards/reports |

## 4.1 RBAC

- Roles must be tenant-scoped or platform-scoped according to role type.
- Project and environment permissions must be enforceable independently.
- Read, create, update, delete/archive, execute, approve, heal, administer and audit must be separate permissions.
- AI actions require both functional permission and AI-policy permission.
- Critical-test healing and release approval must be permission-controlled.
- Every API must perform server-side authorization; UI hiding is not security.

# 5. Global UX/UI Specification

The product should visually resemble a quality command center rather than a traditional test-script IDE. The source blueprint explicitly calls for an AI QA Command Center, One-Click Testing, AI QA Copilot, Test Explorer, Evidence Explorer and Governance Dashboard. filecite turn9file2

## 5.1 Global application shell

| Area | Behavior |
|---|---|
| Top bar | Tenant/project/environment selector, global search, notifications, help, profile |
| Left navigation | Command Center, One-Click Test, Requirements, Application, Tests, Data, Execution, Failures, Healing, Regression, Evidence, Defects, Reports, Integrations, Governance |
| Context header | Current project, environment, release/build and health status |
| Global search | Search requirements, tests, defects, executions, evidence, objects and AI conversations |
| Action drawer | Contextual actions such as generate, execute, heal, create defect, export |
| Activity stream | Recent executions, failures, approvals, integrations and AI actions |
| AI indicator | Shows when an action is AI-assisted/autonomous and exposes confidence/evidence |
| Permission state | Disabled controls must explain required permission rather than silently failing |

## 5.2 Common screen states

- Loading: skeleton/step indicator; no misleading empty state.
- Empty: explain why there is no data and provide next action.
- Success: show result, traceability and next recommended action.
- Partial: show completed and failed sub-operations separately.
- Validation error: field-level and actionable.
- System error: correlation ID and retry path.
- Permission denied: explicit permission required.
- Stale data: indicate AppGraph/test/environment version and offer refresh.
- AI waiting: show reasoning stage without exposing hidden chain-of-thought; show concise rationale, confidence and evidence references.
- Approval required: pause workflow and show exact proposed action, impact, confidence and approver action.

# 6. All Supported User Input Modes

| Input mode | Example | Internal processing |
|---|---|---|
| Natural-language QA objective | "Test customer onboarding including invalid email and duplicate customer." | Intent parser → requirement/test design → TestIntent |
| Application URL | https://customer.example.com | Environment validation → discovery → AppGraph |
| Requirement document | PDF/DOCX/TXT/CSV or pasted text | File service → extraction → requirement intelligence |
| User story | As a customer, I want to… | Requirement parser → acceptance criteria → scenarios |
| API specification | OpenAPI/Swagger/GraphQL/SOAP definition | API parser → endpoint model → API tests |
| Existing test code | Selenium/Playwright project or test artifact | Importer → parser/registration → adapter execution |
| Existing test cases | CSV/XLS/JSON | Importer → validation → test library |
| Database connection | Managed connection reference | Secrets resolver → DB adapter |
| CI/CD event | Commit/PR/build webhook | Integration service → project resolution → impact/risk → execution |
| Manual defect report | Form, screenshot/video/logs | Defect service → duplicate/triage intelligence |
| Exploratory session | Charter + application | Session runner → evidence → defect capture |
| AI Copilot command | Natural-language question/action | Intent router → read/action tool → response/approval |
| Bulk upload | Multiple files/assets | Batch ingestion → validation → job queue |

# 7. Complete User Story → QA Lifecycle

A user story is treated as a controlled quality object rather than only text. The platform must preserve the original story, normalized requirements, acceptance criteria, generated scenarios, tests, automation, executions, defects and evidence.

## 7.1 Intake

User creates story or imports from Jira/Azure DevOps/CSV/document. Required metadata: story key, title, description, business value, priority, epic/feature, acceptance criteria, dependencies, component, owner, release/sprint.

## 7.2 Normalization

Requirement Intelligence extracts actors, actions, objects, business rules, constraints, validations, dependencies and acceptance criteria. Original content remains immutable.

## 7.3 Quality analysis

Detect ambiguity, missing acceptance criteria, conflicting rules, duplicate stories, untestable statements, missing negative paths and dependency gaps.

## 7.4 Test design

Generate positive, negative, boundary, validation, exception, authorization, data, integration and regression scenarios according to selected coverage profile.

## 7.5 Human review

QA Engineer reviews/edits generated scenarios. Approved scenarios become versioned test cases/TestIntent.

## 7.6 Data design

Generate/link datasets, boundary values, invalid values, PII-safe synthetic data and environment-specific data.

## 7.7 Automation

Generate Playwright/Selenium/API automation from TestIntent or attach existing automation.

### 7.8 Execution

Run manually, scheduled, on demand or through CI/CD. Capture full evidence.

### 7.9 Failure analysis

FailureIQ classifies failure and determines whether application, test, data, environment, network, auth, performance or flakiness is responsible.

### 7.10 Healing

AutoHeal proposes safe repair for eligible automation failures. High-confidence actions follow policy; lower-confidence changes require approval.

### 7.11 Defect

If application defect is suspected, AI prepares a defect package; user can review and create/sync externally.

### 7.12 Closure

Defect/test/story links are updated; evidence and approvals are retained.

### 7.13 Release

RiskRun and Release Intelligence recalculate release risk and quality gates.

### 7.14 Learning

Validated outcomes feed QA Knowledge Graph, selector memory, flaky history and defect intelligence.

# 8. Module 01 — Requirements & User Story Management

## 8.1 Screen layout

| Region | Contents |
|---|---|
| Header | Story key/title, status, priority, release, owner, source |
| Left panel | Requirement tree: Epic → Feature → Story → Acceptance Criteria |
| Center | Description, acceptance criteria, business rules, dependencies |
| Right panel | AI quality score, ambiguity warnings, linked tests, defects, coverage |
| Bottom tabs | Versions, traceability, activity, comments, evidence |

## 8.2 Input fields

- External key
- Title
- Description
- Actor/persona
- Business value
- Priority
- Severity/risk
- Epic/feature
- Sprint/release
- Acceptance criteria
- Business rules
- Dependencies

- Assumptions
- Out-of-scope notes
- Attachments
- Source system
- Owner
- Tags

## 8.3 Internal processing

1. Persist raw source/version.
2. Extract structured entities.
3. Create embeddings for semantic search.
4. Run ambiguity/completeness analysis.
5. Identify duplicate/similar requirements.
6. Map requirements to application modules when AppGraph exists.
7. Create candidate test scenarios.
8. Update traceability graph.
9. Emit RequirementCreated/RequirementUpdated events.

## 8.4 Acceptance criteria

- Original story must remain recoverable.
- AI-generated interpretations must be distinguishable from user-authored content.
- No AI-generated business rule may be treated as authoritative without user/source evidence.
- Every test linked to a requirement must retain version linkage.
- Requirement coverage must show tested/untested/partially tested state.

# 9. Module 02 — Application Registration, Discovery & AppGraph™

AppGraph™ is the persistent application model: pages/routes, UI components, objects, workflows, relationships, dependencies and historical changes. The source architecture places application intelligence and AppGraph in the knowledge/intelligence layers. filecite turn8file0

## 9.1 User flow

10. Register application.
11. Select environment.
12. Select credential reference.
13. Set crawl scope and exclusions.
14. Choose browser.
15. Start discovery.
16. Observe progress.
17. Review discovered pages/objects/workflows.
18. Approve or refresh AppGraph version.
19. Link application components to requirements/tests.

## 9.2 Discovery input

| Input | Required | Validation |
|---|---|---|
| Application URL | Yes for web | HTTPS/HTTP allowed by policy; valid hostname |
| Environment | Yes | Environment exists and is accessible |
| Credential reference | Conditional | Secret reference only; never raw password in request logs |
| Scope | Yes | Allowed domains/routes |
| Browser | Yes | Supported browser |
| Depth/page limit | No | Policy bounded |
| Discovery policy | Yes | Crawl, evidence and network policy |

## 9.3 Discovery processing pipeline

20. Launch isolated browser worker.
21. Navigate within policy scope.
22. Capture URL/route, title, DOM, accessibility tree, screenshots and network metadata.
23. Extract interactive objects and semantic properties.
24. Generate locator candidates.
25. Identify forms, tables, menus, dialogs and workflows.
26. Detect API/network dependencies where permitted.
27. Hash structural representations to identify changes.
28. Persist AppGraph version.
29. Generate embeddings for semantic object retrieval.
30. Publish discovery completion and change events.

## 9.4 Object intelligence fields

- object_id
- page_id
- component_type
- semantic_role
- label/text
- aria attributes
- name
- id
- data-testid
- class
- CSS
- XPath
- relative relationships
- frame/shadow root context
- visibility
- enabled state
- DOM fingerprint
- visual fingerprint
- last_seen
- success_rate
- healing_history

# 10. Module 03 — Test Case, Suite, Plan & Library Management

## 10.1 Test case lifecycle

Draft → AI Generated/Imported → Review → Approved → Active → Deprecated/Archived. Versioning is immutable; edits create a new version.

## 10.2 Test case fields

- Test ID/key
- Name
- Objective
- Preconditions
- Actor
- Priority
- Severity
- Test type
- Requirement links
- Business process
- Steps
- Expected result
- Test data
- Environment
- Tags
- Owner
- Automation status
- TestIntent
- Automation artifact
- Version
- Approval state
- Created/updated timestamps

## 10.3 Test types

- Functional
- End-to-End
- Regression
- Smoke
- Sanity
- Negative
- Boundary
- Validation
- Exception
- Authorization
- Data-driven
- Integration
- Component
- Visual

- API
- Database
- Mobile
- Performance
- Security
- Accessibility
- Exploratory
- UAT

## 10.4 Suite/plan features

- Collections and folders
- Ordering/dependencies
- Parameterized execution
- Browser matrix
- Environment matrix
- Priority
- Tags
- Owner
- Schedule
- Quality gate policy
- Retry policy
- Evidence policy
- Approval requirements

# 11. Module 04 — TestIntent™ & AI Test Design

TestIntent™ is the canonical technology-independent representation of what a test means. Generated automation is an implementation artifact. This separation is a core source requirement. filecite turn6file3

## 11.1 TestIntent structure

```
{
  "test_intent_id": "uuid",
  "version": 3,
  "objective": "Verify successful customer onboarding",
  "actors": ["customer"],
  "preconditions": ["customer is authenticated"],
  "steps": [
    {"action":"NAVIGATE","target":"Onboarding"},
    {"action":"ENTER","target":"Email","value_ref":"dataset.valid_email"},
    {"action":"CLICK","target":"Continue"},
    {"action":"VERIFY","target":"Success message","assertion":"visible"}
  ],
  "expected_outcomes":["Customer is created"],
  "negative_variants":[],
  "data_refs":[],
  "requirement_refs":[],
  "adapter_constraints":[],
  "confidence":0.96
}
```

## 11.2 AI test design processing

31. Retrieve requirement context.
32. Retrieve relevant AppGraph objects/workflows.
33. Retrieve historical similar tests and failures.
34. Generate scenario candidates.
35. Classify coverage dimensions.
36. Score relevance/duplication.
37. Generate TestIntent.
38. Validate schema and traceability.
39. Present review diff to user.
40. Persist only after approval or policy-defined auto-approval.

# 12. Module 05 — Test Data Management

- AI test data generation
- Synthetic data
- Masking/anonymization
- Subsetting
- Versioning
- Datasets/data pools
- Dynamic data
- Environment-specific data
- Provisioning
- Cleanup/reset
- Referential integrity
- PII detection
- Parameterization
- Secure vault
- Audit/compliance

## 12.1 Data generation controls

| Data class | Examples | Rules |
|---|---|---|
| Valid | Correct email, valid account, supported amount | Must satisfy business constraints |
| Invalid | Malformed email, unsupported type | Should trigger expected validation |
| Boundary | Min/max length, numeric min/max, date edges | Generate inclusive/exclusive boundary cases |
| Null/empty | Missing required values | Verify required-field handling |
| Duplicate | Existing customer/order/reference | Verify uniqueness rules |
| Security | Injection strings, unauthorized roles | Only run in authorized test environments |
| PII-like | Names, phone, addresses | Use synthetic/masked values |

# 13. Module 06 — Complete Testing Engine

# ONECLICKQA – COMPLETE FEATURE LIST

AI-Native Autonomous QA Platform – End to End Quality Engineering

## 1. TESTING CAPABILITIES

### 1.1 WEB & UI TESTING

- Cross-Browser Testing
- Parallel Browser Execution
- Responsive Testing
- UI Functional Testing
- End-to-End Testing
- Regression Testing
- Smoke / Sanity Testing
- Visual Regression Testing
- Component Testing
- Form Validation Testing
- Screenshot / Video Capture
- Console & Network Logs
- Auto-Wait & Smart Sync
- Drag & Drop, JS Alerts, Frames
- Shadow DOM Support

### 1.2 API TESTING

- REST API Testing
- GraphQL Testing
- SOAP API Testing
- API Functional Testing
- Schema Validation
- Contract Testing
- Data-Driven API Testing
- Authentication (OAuth, JWT, API Key, Basic, Digest)
- Header / Parameter Validation
- Response Time Validation
- Rate Limiting & Throttling
- Chaining & Dependency
- Mock Service Virtualization
- API Security Testing

### 1.3 DATABASE TESTING

- Database Connectivity
- CRUD Operation Testing
- Data Integrity Validation
- Data Consistency Checking
- UI-DB Reconciliation
- API-DB Reconciliation
- Stored Procedure Testing
- Complex Query Validation
- Data Comparison
- Row Count Verification
- Transaction Validation
- Data Seeding & Cleanup
- Multiple DB Support (MySQL, PostgreSQL, SQL Server, Oracle, MongoDB)

### 1.4 MOBILE TESTING

- Android App Testing (Appium)
- iOS App Testing (Appium)
- Hybrid App Testing
- Mobile Web Testing
- Native App Testing
- Parallel Mobile Execution
- Real Device & Emulator Support
- Gestures (Swipe, Pinch, Rotate)
- Push Notification Testing
- Background / Foreground
- Device Info & Permissions
- Network Simulation
- Location / GPS Mocking
- Mobile Visual Testing

### 1.5 PERFORMANCE TESTING

- Load Testing
- Stress Testing
- Spike Testing
- Endurance Testing
- Scalability Testing
- Volume Testing
- Concurrency Testing
- Response Time Analysis (p50, p90, p95, p99)
- Throughput Analysis
- Error Rate Analysis
- Resource Utilization
- Capacity Planning
- Custom Performance Metrics
- k6 / JMeter Integration

### 1.6 SECURITY TESTING

- DAST Scanning (OWASP ZAP)
- SAST Scanning (Semgrep, SonarQube)
- API Security Testing
- OWASP Top 10 Checks
- Authentication Testing
- Authorization Testing
- Session Management Testing
- Input Validation Testing
- SQL Injection Testing
- XSS Testing
- CSRF Testing
- Security Misconfiguration Checks
- Vulnerability Reporting
- Risk Rating & Remediation

### 1.7 ACCESSIBILITY TESTING

- WCAG 2.1 / 2.2 Compliance
- Automated Accessibility Scan (axe-core)
- Color Contrast Validation
- ARIA Validation
- Keyboard Navigation Testing
- Screen Reader Compatibility
- Alt Text & Labels Validation
- Accessibility Issue Reporting
- Severity Classification
- AI-Based Accessibility Insights
- Accessibility Score & Trend

### 1.8 VISUAL TESTING

- Screenshots Capture
- Baseline Management
- Visual Regression Comparison
- Layout Shift Detection
- Element Level Comparison
- AI-Based Visual Analysis
- Threshold Configuration
- Cross Browser Visual Testing
- Responsive Visual Testing
- Visual Diff Reports
- Change Highlighting

### 1.9 SERVICE VIRTUALIZATION & MOCKING

- Mock API Creation
- Virtual Service Simulation
- Dynamic Response Mocking
- Scenario Based Mocking
- Delay / Error Simulation
- Contract Based Mocking
- Environment Isolation
- Record & Playback
- Data Masking in Mocks

### 1.10 EXPLORATORY & UAT TESTING

- AI-Guided Exploratory Testing
- Session Based Exploratory
- Charter Management
- Bug Capture During Exploration
- UAT Test Management
- Business Scenario Testing
- Step Logging & Evidence
- Feedback & Approvals
- Sign-off & Traceability

## 2. INTELLIGENT QA ENGINE

- 2.1 Requirement Intelligence (NLP)
- 2.2 Application Discovery & AppGraph™
- 2.3 Object Intelligence Repository
- 2.4 Test Design & TestIntent™
- 2.5 AI Test Generation
- 2.6 AI Test Data Generation
- 2.7 FailureIQ™ (Failure Intelligence)
- 2.8 AutoHeal™ (Self-Healing Automation)
- 2.9 RiskRun™ (Risk Based Regression)
- 2.10 Flaky Test Detection
- 2.11 Impact Analysis & Change Intelligence
- 2.12 Smart Wait & Synchronization
- 2.13 Self-Learning Engine
- 2.14 QA Knowledge Graph
- 2.15 Predictive Quality Analytics

## 3. TEST MANAGEMENT

- 3.1 Test Case Management
- 3.2 Test Suite Management
- 3.3 Test Planning & Scheduling
- 3.4 Tags, Labels & Metadata
- 3.5 Requirement Traceability
- 3.6 Business Process Mapping
- 3.7 Versioning & Baselines
- 3.8 Reusable Components
- 3.9 Test Library Management
- 3.10 Custom Fields & Attributes
- 3.11 Test Case Review & Approval
- 3.12 Test Case Reusability Analysis
- 3.13 Test Case Health Dashboard
- 3.14 Bulk Operations
- 2.15 Import / Export (XLS, CSV, JSON)

## 4. TEST DATA MANAGEMENT

- 4.1 AI Test Data Generation
- 4.2 Synthetic Data Generation
- 4.3 Data Masking & Anonymization
- 4.4 Data Subsetting
- 4.5 Data Versioning
- 4.6 Data Pools & Datasets
- 4.7 Dynamic Data Handling
- 4.8 Environment Specific Data
- 4.9 Data Provisioning
- 4.10 Data Cleanup & Reset
- 4.11 Referential Integrity
- 4.12 PII Detection & Protection
- 4.13 Data Parameterization
- 4.14 Secure Data Vault
- 4.15 Audit & Compliance for Data

## 5. EXECUTION MANAGEMENT

- 5.1 One-Click Test Execution
- 5.2 Parallel Execution (Tests)
- 5.3 Parallel Browsers & Matrix Execution
- 5.4 Execution Queue & Scheduler
- 5.5 Priority & Resource Allocation
- 5.6 Retry Mechanism
- 5.7 Failure Handling Rules
- 5.8 Execution Timeouts
- 5.9 Dependency Handling
- 5.10 Dynamic Environment Selection
- 5.11 Worker Management
- 5.12 Distributed Execution (Grid/Cloud)
- 5.13 Real-Time Execution Monitoring
- 5.14 Resource Utilization Monitoring
- 5.15 Cancellation & Stop Conditions

## 6. CI/CD & DEVOPS INTEGRATION

- 6.1 Git Integration (GitHub, GitLab, Bitbucket)
- 6.2 Jenkins Integration
- 6.3 Azure DevOps Integration
- 6.4 GitHub Actions Integration
- 6.5 GitLab CI/CD Integration
- 6.6 Trigger on Commit / PR
- 6.7 Scheduled Pipeline Execution
- 6.8 Quality Gates & Policies
- 6.9 Build Status & Reporting
- 6.10 Artifact Management
- 6.11 Slack / Teams Notifications
- 6.12 Webhook Integrations
- 6.13 Containerized Execution
- 6.14 Helm Chart Deployment
- 6.15 Infrastructure as Code Support

## 7. REPORTING & ANALYTICS

- 7.1 Execution Dashboard (Real-Time)
- 7.2 Test Summary Reports
- 7.3 Defect Reports
- 7.4 Requirement Coverage Reports
- 7.5 Trend Analysis
- 7.6 Flaky Test Reports
- 7.7 Performance Reports
- 7.8 Security Reports
- 7.9 Accessibility Reports
- 7.10 Visual Comparison Reports
- 7.11 Release Readiness Report
- 7.12 Executive Dashboard (CIO/CTO)
- 7.13 Custom Report Builder
- 7.14 Scheduled Reports
- 7.15 Data Export (PDF, XLS, CSV, JSON)

## 8. COLLABORATION & WORKFLOW

- 8.1 Role Based Access Control (RBAC)
- 8.2 User Management
- 8.3 Team Management
- 8.4 Permission Management
- 8.5 Approval Workflows
- 8.6 Commenting & Discussions
- 8.7 @Mentions & Notifications
- 8.8 Activity Feed
- 8.9 Audit Logs
- 8.10 Change History
- 8.11 Review & Sign-off
- 8.12 Defect Triage Workflow
- 8.13 SLA & Escalation Rules
- 8.14 Custom Workflow Builder
- 8.15 Multi-Tenancy Support

## 9. GOVERNANCE & SECURITY

- 9.1 SSO (SAML, OAuth, OIDC, LDAP)
- 9.2 MFA Support
- 9.3 Secrets Management
- 9.4 API Key Management
- 9.5 Encryption (In-Transit, At-Rest)
- 9.6 Data Retention Policies
- 9.7 Compliance (GDPR, SOC2, ISO)
- 9.8 Audit Trail & Logging
- 9.9 IP Whitelisting
- 9.10 Policy Based AI Usage
- 9.11 AI Governance & Guardrails
- 9.12 Usage Quotas & Limits
- 9.13 Data Residency Control
- 9.14 Backup & Restore
- 9.15 Disaster Recovery

## 10. PLATFORM SERVICES

**10.1 API MANAGEMENT**
API Gateway, rate limiting, versioning, API documentation

**10.2 NOTIFICATION SERVICE**
Email, SMS, Slack, Teams, Webhook notifications

**10.3 FILE & ARTIFACT SERVICE**
Upload, download, versioning, artifact storage

**10.4 SEARCH SERVICE**
Global search across tests, requirements, defects, evidence

**10.5 CONFIGURATION SERVICE**
Global & project configs, execution settings, templates

**10.6 OBSERVABILITY**
Logs, metrics, tracing, health checks, alerts

**10.7 SCHEDULER & TRIGGERS**
Cron, event based, webhooks, manual triggers

**10.8 CACHE SERVICE**
Redis based caching for performance & sessions

**10.9 KNOWLEDGE SERVICE**
Vector search, RAG, embeddings, QA knowledge graph

**LEGEND (ICONS USED)**

Web Testing · API Testing · Database · Mobile · Performance · Security · Accessibility · Visual · AI / Intelligence · CI/CD · Collaboration · Governance · Data / Storage

Supplied OneClickQA Complete Feature List reference. It defines ten testing-capability groups plus intelligent QA, management, data, execution, CI/CD, reporting, collaboration, governance and platform services. filecite turn9file15

## 13.1 Web & UI Testing

| Capability | Required behavior / internal processing |
|---|---|
| Cross-browser | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Parallel browser execution | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Responsive testing | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| UI functional testing | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| End-to-end testing | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Regression | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Smoke/Sanity | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Visual regression | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Component testing | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Form validation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Screenshot/video | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Console/network logs | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Auto-wait/smart synchronization | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Drag/drop | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| JavaScript alerts | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Frames | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Shadow DOM | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.2 API Testing

| Capability | Required behavior / internal processing |
|---|---|
| REST | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| GraphQL | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| SOAP | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Functional API | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Schema validation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Contract testing | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Data-driven API | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| OAuth/JWT/API key/Basic/Digest | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Header/parameter validation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Response-time validation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Rate limiting/throttling | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Chaining/dependencies | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Mock service virtualization | Input → parser/adapter → test plan → execution → evidence → result → analytics |

| | |
|---|---|
| API security | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.3 Database Testing

| Capability | Required behavior / internal processing |
|---|---|
| MySQL | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| PostgreSQL | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| SQL Server | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Oracle | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| MongoDB | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| CRUD | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Data integrity | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Consistency | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| UI↔DB reconciliation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| API↔DB reconciliation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Stored procedures | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Complex query validation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Data comparison | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Row count | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Transaction validation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Seeding/cleanup | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.4 Mobile Testing

| Capability | Required behavior / internal processing |
|---|---|
| Android/Appium | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| iOS/Appium | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Hybrid | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Mobile web | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Native | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Parallel devices | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Real device/emulator | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Swipe/pinch/rotate | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Push notifications | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Background/foreground | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Device info/permissions | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Network simulation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| GPS/location mocking | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Mobile visual | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.5 Performance Testing

| Capability | Required behavior / internal processing |
|---|---|
| Load | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Stress | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Spike | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Endurance | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Scalability | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Volume | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Concurrency | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| p50/p90/p95/p99 | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Throughput | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Error rate | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Resource utilization | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Capacity planning | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Custom metrics | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| k6/JMeter | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.6 Security Testing

| Capability | Required behavior / internal processing |
|---|---|
| DAST/OWASP ZAP | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| SAST/Semgrep/SonarQube | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| API security | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| OWASP Top 10 | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Authentication | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Authorization | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Session management | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Input validation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| SQL injection | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| XSS | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| CSRF | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Security misconfiguration | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Vulnerability reporting | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Risk/remediation | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.7 Accessibility Testing

| Capability | Required behavior / internal processing |
|---|---|

| | |
|---|---|
| WCAG 2.1/2.2 | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| axe-core scan | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Color contrast | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| ARIA | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Keyboard navigation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Screen reader compatibility | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Alt text/labels | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Issue reporting | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Severity | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| AI accessibility insights | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Accessibility score/trend | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.8 Visual Testing

| Capability | Required behavior / internal processing |
|---|---|
| Screenshots | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Baseline management | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Visual comparison | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Layout shift | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Element-level comparison | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| AI visual analysis | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Threshold configuration | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Cross-browser | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Responsive | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Visual diff reports | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Change highlighting | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.9 Service Virtualization & Mocking

| Capability | Required behavior / internal processing |
|---|---|
| Mock API creation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Virtual service simulation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Dynamic response | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Scenario-based mocking | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Delay/error simulation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Contract-based mocks | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Environment isolation | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Record/playback | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Data masking | Input → parser/adapter → test plan → execution → evidence → result → analytics |

### 13.10 Exploratory & UAT

| Capability | Required behavior / internal processing |
|---|---|
| AI-guided exploratory | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Session-based exploration | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Charter management | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Bug capture | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| UAT management | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Business scenarios | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Step logging/evidence | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Feedback/approval | Input → parser/adapter → test plan → execution → evidence → result → analytics |
| Sign-off/traceability | Input → parser/adapter → test plan → execution → evidence → result → analytics |

## 13.11 Common test-engine contract

```
TestEngine.execute(
  TestIntent | TestArtifact,
  Environment,
  DataSet,
  ExecutionPolicy
) -> ExecutionResult

ExecutionResult:
  status
  step_results[]
  assertions[]
  metrics
  artifacts[]
  failures[]
  evidence_refs[]
  environment_snapshot
  adapter_version
  worker_metadata
```

All test engines must expose a consistent result contract so reporting, FailureIQ, EvidenceGraph and release intelligence do not depend on the underlying tool.

# 14. Module 07 — Execution Management & Execution Fabric

## 14.1 Execution request

| Field | Required | Description |
|---|---|---|
| Project | Yes | Execution scope |
| Tests/suite/plan | Yes | What to execute |
| Environment | Yes | Target environment |
| Adapter | Yes | Playwright/Selenium/API/etc. |
| Browser/device matrix | Conditional | Target execution matrix |
| Priority | Yes | Queue priority |
| Concurrency | No | Parallel worker limit |

| | | |
|---|---|---|
| Retry policy | No | Controlled retries |
| Timeout policy | No | Test/step timeout |
| Evidence policy | Yes | Artifact levels |
| Healing policy | No | Whether healing is allowed |
| Failure policy | No | Strict/relaxed and handling |
| Quality gate | No | CI/release decision policy |

## 14.2 Internal execution flow

41. API receives request and validates permission.
42. Create workflow/execution record.
43. Resolve dependencies and execution plan.
44. Push jobs to Redis/event queue.
45. Worker obtains lease.
46. Worker resolves secrets just in time.
47. Create isolated runtime/browser context.
48. Execute adapter actions.
49. Emit step events.
50. Capture artifacts.
51. Persist results.
52. Invoke FailureIQ on failures.
53. Invoke AutoHeal if eligible.
54. Re-run if healing applied.
55. Aggregate results.
56. Update EvidenceGraph.
57. Evaluate quality gate/release risk.
58. Publish completion event/notification.

## 14.3 Evidence levels

| Level | Artifacts |
|---|---|
| Minimum | Status, timings, error, console summary |
| Standard | Screenshots, DOM, console, network metadata, logs |
| Diagnostic | Video, HAR, traces, full DOM, request/response, runtime metadata |
| AI analysis | Failure classification, root cause summary, confidence, evidence references |
| Compliance | Full traceability, approvals, versions, actor/audit records |

# 15. Module 08 — FailureIQ™

FailureIQ must decide whether a failure is an application defect, test defect, locator failure, test-data failure, environment failure, network failure, authentication failure, performance failure, flaky test or unknown/human-review case. The source also identifies granular error codes such as ELEMENT_NOT_FOUND, ASSERTION_FAILURE, TIMEOUT_ERROR, NAVIGATION_ERROR, network/console errors, EXECUTION_ERROR, INVALID_INPUT, PLAYWRIGHT_ENV_ERROR and RUNNER_CRASH.

filecite turn6file6

## 15.1 Failure analysis input

- Execution ID

- Failed step
- Exception/error
- Screenshot
- Video/trace
- DOM snapshot
- Locator candidates
- Console
- Network/HAR
- API request/response
- Database result if applicable
- Environment snapshot
- Build/commit
- Test version
- Historical similar failures
- Previous defect outcomes

## 15.2 Diagnosis pipeline

59. Normalize raw error.
60. Collect correlated evidence.
61. Check deterministic rules first.
62. Retrieve similar historical failures.
63. Check environment health.
64. Check network/auth status.
65. Compare DOM/object fingerprint.
66. Check test-data validity.
67. Run flaky correlation.
68. Use LLM only when deterministic evidence is insufficient or synthesis is needed.
69. Return classification + confidence + concise rationale + evidence references.
70. Create recommended next action.

## 15.3 Failure output

```
{
  "classification":"LOCATOR_FAILURE",
  "sub_classification":"ELEMENT_NOT_FOUND",
  "confidence":0.97,
  "root_cause_summary":"The original selector no longer resolves.",
  "recommended_action":"AUTO_HEAL",
  "evidence_refs":["ev-1","ev-2"],
  "similar_failures":["failure-182"],
  "defect_candidate":false,
  "requires_human_review":false
}
```

# 16. Module 09 — AutoHeal™

## 16.1 Healing signals

| Signal | Priority | Use |
|---|---|---|
| Accessibility role/name | Highest | Semantic target match |

| | | |
|---|---|---|
| Stable semantic attributes | High | data-testid/name/aria-label |
| Stable ID | High | Exact identity match |
| Stable CSS | High | Structural match |
| Relative locator | Medium | Relationship-based match |
| XPath | Medium | Fallback structural match |
| DOM similarity | Medium | Contextual match |
| Visual similarity | Fallback | Confirmation/last-resort candidate |

## 16.2 Confidence policy

>95%: eligible for automatic healing only when project policy permits. 80–95%: suggest repair and require approval unless policy explicitly permits. <80%: human review. Every applied repair must be re-executed and validated. Critical tests must never be silently changed. This confidence policy is explicitly specified in the prior project requirements. filecite turn6file6

## 16.3 Healing record

- Original locator
- Candidate locators
- Replacement locator
- Signal scores
- Confidence
- Model/algorithm version
- Reason
- Approver
- Applied timestamp
- Validation execution
- Result
- Rollback reference

# 17. Module 10 — RiskRun™, Impact Analysis & Regression Intelligence

RiskRun maps code/configuration/requirement changes to impacted components and workflows, uses business criticality, production usage, historical failures and defect frequency, then selects high-value tests and produces a parallelizable plan. filecite turn7file10

## 17.1 Risk inputs

- Git diff/commit/PR
- Changed files/modules
- Requirement changes
- Application component relationships
- Business criticality
- Production usage
- Historical defects
- Historical failures
- Test failure frequency
- Flakiness
- Release type
- Environment health

## 17.2 Risk score model

Proposed normalized score: Risk = weighted(Change Impact + Business Criticality + Historical Failure Signal + Defect Signal + Coverage Gap + Environment Signal + Flaky Uncertainty). Weights must be configurable and versioned. The final score must always display contributing factors.

## 17.3 Regression output

- Impacted requirements
- Impacted components
- Critical tests
- Recommended test suite
- Tests safe to skip
- Parallelization groups
- Estimated duration
- Risk score
- Quality gate recommendation

# 18. Module 11 — EvidenceGraph™

EvidenceGraph connects requirement version → acceptance criteria → test scenario → TestIntent → automation → execution → environment/browser → test data → screenshot/video/DOM/console/network → AI analysis → defect → approval history.
filecite turn6file6

## 18.1 Evidence node types

- Requirement
- Acceptance criterion
- Test case
- TestIntent
- Automation artifact
- Execution
- Step result
- Environment
- Dataset
- Screenshot
- Video
- DOM
- Network
- Console
- API payload
- DB result
- Failure
- AI analysis
- Healing
- Defect
- Approval
- Release assessment

## 18.2 Evidence explorer UI

| Panel | Behavior |
|---|---|
| Graph | Interactive traceability chain |
| Timeline | Chronological execution/evidence events |
| Artifact viewer | Screenshot/video/log/DOM/network/API/DB views |
| AI panel | Classification, confidence, rationale summary and evidence links |
| Defect panel | Linked defect and external synchronization |
| Audit panel | Actors, approvals, changes and timestamps |

# 19. Module 12 — Defect / Bug Management

OneClickQA must support defects created manually by users, automatically from execution failures, suggested by FailureIQ/AI Defect Intelligence, captured during exploratory/UAT sessions, imported from external ALM/defect systems, or synchronized from CI/CD quality-gate failures.

## 19.1 Ways a bug can be raised

| Source | Behavior |
|---|---|
| Manual defect | User clicks Create Defect from Defects module or any test/execution screen. |
| From failed execution | User selects Create Defect from a failed execution; system pre-populates evidence. |
| AI suggested defect | FailureIQ classifies as likely application defect and prepares draft. |
| Auto defect policy | Configured critical failures can create draft or external defect automatically; final auto-submit is policy controlled. |
| Exploratory testing | Tester captures bug during an exploratory session with current screen/evidence. |
| UAT | Business user reports issue against UAT scenario/requirement. |
| API testing | Failed API assertion creates defect with request, response, headers and schema evidence. |
| Database testing | Reconciliation/integrity failure creates defect with query/result/difference evidence. |
| Visual testing | Visual threshold breach can create a defect with baseline/current/diff images. |
| Accessibility testing | Accessibility violation can create a defect with rule, selector, screenshot and WCAG reference. |
| Security testing | Vulnerability finding can create a security defect with finding, severity and remediation. |
| Performance testing | Threshold breach can create performance defect with p95/throughput/resource evidence. |
| CI/CD quality gate | Failed quality gate can create a release/build defect or incident according to policy. |
| External synchronization | Imported Jira/Azure DevOps/ServiceNow/Bugzilla defect is mapped into local defect model. |

## 19.2 Mandatory defect information

| Group | Required information |
|---|---|
| Identity | Defect ID, external ID, project, component/module, title |
| Classification | Defect type, category, source, detected-by, suspected cause |
| Severity | Critical/Blocker, High, Medium, Low or configurable |
| Priority | P0/P1/P2/P3 or configurable |
| Status | New, Triaged, Assigned, In Progress, Fixed, Ready for Retest, Verified, Reopened, Rejected, Duplicate, Deferred, Closed |
| Environment | Environment, browser/device, OS, version, region |
| Build/release | Build number, commit SHA, branch, release, deployment version |
| Requirement | Requirement/story/acceptance criterion |
| Test | Test case/version, suite, TestIntent, automation artifact |

| | |
|---|---|
| Preconditions | Required state/data/permissions |
| Steps | Ordered reproducible steps |
| Expected | Expected behavior/result |
| Actual | Observed behavior/result |
| Reproducibility | Always/intermittent/once; frequency |
| Evidence | Screenshots, video, DOM, logs, console, network/HAR, API request/response, DB result, visual diff, performance metrics |
| Technical context | Exception, stack trace, locator, endpoint, query, service/component |
| AI analysis | Classification, confidence, root-cause summary, recommendation, similar defects |
| Assignment | Assignee, team, watcher |
| Labels | Tags, component, sprint, release |
| SLA | Target response/fix dates, escalation |
| Relations | Duplicate/blocker/blocked-by/related/parent-child |
| Security | Security flag, confidentiality, vulnerability category where applicable |
| Audit | Reporter, created/updated timestamps, history, approvals |

## 19.3 Defect creation workflow

71. Collect source context automatically.
72. Run duplicate similarity check.
73. Generate draft title/description if AI-assisted.
74. Populate environment/build/test/evidence automatically.
75. Recommend severity and priority with confidence.
76. User reviews mandatory fields.
77. User chooses Create Local / Create External / Save Draft.
78. External connector maps fields and stores external ID.
79. Defect link is added to EvidenceGraph.
80. Notify assignee/watchers.
81. Audit event is recorded.

## 19.4 Duplicate detection

Use exact external ID, normalized title similarity, semantic embeddings, linked requirement/component, stack/endpoint/locator signatures and historical defects. Duplicate suggestions must never silently merge records; a user must confirm merge/duplicate state.

# 20. Module 13 — Dashboards, Analytics & Release Intelligence

| Report | Required content |
|---|---|
| Command Center | Pass/fail, coverage, risk, defects, flaky, healing, worker health |
| Test Summary | Execution totals, duration, test status, environment/browser breakdown |
| Defect Report | Severity/priority, aging, reopened, leakage, root cause |
| Requirement Coverage | Covered/partially/untested requirements and acceptance criteria |
| Trend Analysis | Quality, failure, coverage, defect, automation trends |
| Flaky Report | Flaky tests, frequency, stability, suspected causes |
| Performance Report | p50/p90/p95/p99, throughput, error/resource metrics |
| Security Report | Findings, severity, risk, remediation |
| Accessibility Report | WCAG/axe findings and trend |
| Visual Report | Baseline/current/diff/threshold changes |
| Release Readiness | Risk 0–100, gates, critical defects, coverage, evidence completeness |
| Executive Dashboard | Quality trend, release risk, ROI, autonomous QA coverage |
| Custom Report Builder | Selectable metrics, filters, grouping and export |
| Scheduled Reports | Email/Slack/Teams/webhook schedules |
| Exports | PDF, XLS, CSV, JSON |

## 20.1 Release readiness workflow

82. Collect current build/release context.
83. Run RiskRun impact analysis.
84. Calculate critical requirement coverage.
85. Check critical test completion.
86. Evaluate critical/high defects.
87. Evaluate flaky-test policy.
88. Evaluate environment health.
89. Calculate risk score.
90. Apply quality-gate policy.
91. Generate recommendation: GO / GO WITH RISK / NO-GO / INCONCLUSIVE.
92. Request approval when configured.
93. Persist signed release assessment.

# 21. Module 14 — Integrations & CI/CD

| Integration family | Systems |
|---|---|
| ALM/Test Management | Jira, Azure DevOps, TestRail, HP ALM target |
| Defect Management | Jira, ServiceNow, Bugzilla |
| CI/CD | Jenkins, GitHub Actions, GitLab CI, Azure DevOps |
| Source Control | GitHub, GitLab, Bitbucket, Azure Repos |
| ChatOps | Slack, Microsoft Teams, Email, Webhooks, SMS where configured |
| Identity | SAML, OAuth/OIDC, LDAP |
| Cloud | AWS, Azure, GCP |
| QA/Security | Postman, OWASP ZAP, BrowserStack |
| Monitoring | Datadog, New Relic |

## 21.1 CI/CD processing

94. Webhook received and signature validated.
95. Map repository/branch to tenant/project.
96. Resolve environment.
97. Resolve changed files/build metadata.
98. RiskRun impact analysis.
99. Select tests.
100. Queue parallel execution.
101. Stream result to CI/CD.
102. Analyze failures.
103. Evaluate quality gate.
104. Publish status/check.
105. Attach report/artifacts.
106. Block or permit deployment according to policy.

# 22. Module 15 — Platform Services

| Service | Detailed responsibility |
|---|---|
| API Management | Gateway, authentication, authorization, rate limiting, versioning, OpenAPI |
| Notification | Email, SMS, Slack, Teams, webhook templates, retries |
| File & Artifact | Upload/download/versioning, signed URLs, lifecycle, checksums |
| Search | Global search over requirements/tests/defects/evidence |
| Configuration | Tenant/project/environment/execution/AI templates |

| | |
|---|---|
| Scheduler & Triggers | Cron, event, webhook, manual |
| Cache | Redis sessions/cache/queues |
| Knowledge | Vector search, RAG, embeddings, QA knowledge graph |
| Audit | Immutable audit event capture and search |
| Observability | Logs, metrics, traces, health, alerts |
| Secrets | Vault/KMS/managed secret references |
| Feature Flags | Progressive rollout and rollback for risky autonomy |

# 23. Module 16 — AI QA Copilot

## 23.1 Supported intents

- Investigate: what failed, why, what changed.
- Query: show coverage, defects, risky tests, untested requirements.
- Generate: create test cases/data/scenarios.
- Execute: run a test/suite/regression plan.
- Heal: inspect or request healing.
- Evidence: show evidence for a requirement/test/defect.
- Release: assess release readiness.
- Explain: explain a failure, quality trend or risk score.
- Navigate: open relevant module/entity.

## 23.2 Copilot processing

107. Classify user intent.
108. Resolve project/tenant context.
109. Retrieve relevant entities and evidence.
110. Check permissions.
111. Build bounded tool request.
112. Run read-only tool directly or require approval for mutations.
113. Return concise result with entity links, confidence where relevant and evidence references.
114. Persist AI run metadata without storing unnecessary sensitive prompt data.

# 24. Module 17 — AI Agent Suite & Internal Processing

| Agent | Input | Output | Processing |
|---|---|---|---|
| Requirement Intelligence Agent | Requirements/documents/user stories | Structured requirements, rules, acceptance criteria, gaps | Extraction → normalization → ambiguity → traceability |
| Application Discovery Agent | URL/environment | AppGraph/pages/objects/workflows | Crawl → inspect → fingerprint → semantic object extraction → graph update |
| Test Design Agent | Requirement + AppGraph | Scenario/test candidates | Coverage model → generation → dedupe → scoring |
| Test Data Agent | TestIntent/rules | Datasets | Constraint extraction → generation → validation → masking |
| Automation Generation Agent | TestIntent/AppGraph | Playwright/Selenium/API artifacts | Adapter selection → code generation → syntax/lint → executable validation |
| Execution Intelligence Agent | Plan/environment | Execution schedule/results | Dependency sort → queue → allocation → retry/stop |
| Failure Intelligence Agent | Failure/evidence/history | Classification/root cause | Evidence normalization → deterministic rules → retrieval → AI synthesis |
| AutoHeal Agent | Failure/AppGraph/history | Repair candidates | Locator candidate generation → scoring → policy → validation rerun |

| | | | |
|---|---|---|---|
| Regression Intelligence Agent | Change set/history | Impacted tests/risk plan | Dependency graph → criticality → historical risk → selection |
| Evidence Intelligence Agent | Requirements/executions/artifacts | EvidenceGraph package | Entity linkage → completeness → traceability → audit |

## 24.1 AI Gateway

- Provider abstraction
- Model routing by task
- Token/cost tracking
- Timeout/retry/fallback
- Prompt version reference
- Data policy enforcement
- Model allowlist
- PII/secrets redaction
- Structured-output validation
- Evaluation hooks

## 24.2 Prompt Registry

- Prompt ID/version
- Agent mapping
- Model compatibility
- Variables/schema
- Evaluation dataset
- Approval state
- Performance metrics
- Rollback to prior version

# 25. Module 18 — QA Knowledge Graph & Continuous Learning

The knowledge layer retains historical tests, failures, defects, requirements, application objects and outcomes so future AI decisions use project-specific evidence rather than only generic model knowledge.

## 25.1 Knowledge entities

- Requirement
- Business rule
- Application component
- Object
- Workflow
- Test
- TestIntent
- Automation
- Execution
- Failure
- Defect
- Healing
- Release

- Evidence
- Environment
- Build
- Team/owner

## 25.2 Learning signals

- Validated root cause
- Accepted/rejected healing
- Successful selector
- Failed selector
- Flaky frequency
- Defect recurrence
- Release impact
- Test usefulness
- Human approval/rejection
- Coverage gaps

# 26. Complete Test Management Feature Requirements

- 26.1 Test case management
- 26.2 Test suite management
- 26.3 Test planning and scheduling
- 26.4 Tags/labels/metadata
- 26.5 Requirement traceability
- 26.6 Business process mapping
- 26.7 Versioning/baselines
- 26.8 Reusable components
- 26.9 Test library
- 26.10 Custom fields/attributes
- 26.11 Review/approval
- 26.12 Test case reusability analysis
- 26.13 Test case health dashboard
- 26.14 Bulk operations
- 26.15 Import/export XLS/CSV/JSON

## 26.1 Test health score

Proposed score based on execution stability, age, last update, failure concentration, selector healing history, duplication, requirement relevance, runtime cost and maintenance burden. The formula must be configurable and explainable.

# 27. Module 19 — Collaboration, Workflow & Governance

- RBAC
- User management
- Team management
- Permission management
- Approval workflows
- Comments/discussions

- @mentions/notifications
- Activity feed
- Audit logs
- Change history
- Review/sign-off
- Defect triage workflow
- SLA/escalation
- Custom workflow builder
- Multi-tenancy
- SSO SAML/OAuth/OIDC/LDAP
- MFA
- Secrets
- API keys
- Encryption
- Retention
- GDPR/SOC2/ISO readiness
- IP whitelisting
- AI usage policy
- AI guardrails
- Quotas
- Data residency
- Backup/restore
- Disaster recovery

## 27.1 Approval model

| Action | Default policy |
|---|---|
| Generate tests | Review configurable |
| Execute tests | User permission |
| Create external defect | User confirmation by default |
| Heal >95% | Auto only if project policy allows |
| Heal 80–95% | Approval |
| Heal <80% | Human review |
| Change critical test | Approval mandatory |
| Change AI prompt/model | AI governance approval |
| Release GO/NO-GO | Release role approval when configured |

# 28. Database & Data Model

## 28.1 Storage architecture

| Store | Purpose | Technology direction |
|---|---|---|
| System of record | Transactional entities | PostgreSQL |
| Vector | Embeddings/semantic retrieval | pgvector |
| Cache/queue | Queue, state, sessions | Redis |
| Artifacts | Screenshots/video/logs/reports | S3-compatible object storage |
| Events | Domain events | Redis/RabbitMQ initially; Kafka at scale |
| Search | Global/log search | OpenSearch/ELK as needed |

## 28.2 Core domains/tables

| Domain | Representative tables |
|---|---|
| Identity | tenants, users, teams, roles, permissions, memberships, sessions |
| Project | projects, environments, applications, releases, builds |
| Requirement | requirements, versions, business_rules, acceptance_criteria, dependencies |
| Application | app_versions, pages, components, objects, locators, workflows, network_dependencies |
| Test | test_cases, test_versions, suites, suite_items, plans, test_intents, intent_steps, automation_artifacts |
| Data | datasets, records, pools, policies, provisioning_jobs, cleanup_jobs, data_masks |
| Execution | executions, execution_items, execution_steps, workers, jobs, retries, runtime_snapshots |
| Failure | failures, failure_evidence, analyses, flaky_signals, healing_attempts, selector_memory |
| Risk | regression_analyses, impacted_components, risk_scores, quality_gates, release_assessments |
| Evidence | evidence_packages, evidence_items, graph_edges, artifacts |
| Defect | defects, defect_links, external_defect_links, comments, triage_history |
| AI | ai_runs, ai_actions, prompts, prompt_versions, model_registry, evaluations, policies |
| Integration | integrations, credentials_refs, webhooks, sync_jobs, mappings |
| Governance | audit_logs, approvals, notifications, activity_events |

## 28.3 Common columns

- id UUID
- tenant_id UUID where tenant-owned
- project_id UUID where project-owned
- created_at UTC
- updated_at UTC
- created_by
- updated_by
- version where versioned
- status
- metadata JSONB where flexible metadata is required

## 28.4 Indexing

- tenant_id/project_id composite indexes
- foreign-key indexes
- status/time indexes for execution queues
- requirement/test external keys
- execution_id/step_id
- defect severity/status
- embedding vector indexes
- full-text/search indexes where required

# 29. API Specification

The blueprint explicitly requires API-first architecture and identifies core endpoints for project creation, discovery, test generation/execution/healing, failure analysis, regression analysis, execution/evidence retrieval and defect creation.

filecite turn6file1

## 29.1 API standards

- Base: /api/v1
- JSON by default
- UUID IDs
- UTC ISO-8601
- Bearer/OIDC authentication
- Idempotency-Key for job creation
- X-Correlation-ID
- Cursor pagination for large collections
- OpenAPI generated/documented
- Standard error envelope
- Rate limits
- Audit events for mutations

### Projects

- GET /projects
- POST /projects
- GET /projects/{id}
- PATCH /projects/{id}
- GET /projects/{id}/members
- POST /projects/{id}/members

### Applications

- GET /projects/{id}/applications
- POST /projects/{id}/applications
- POST /applications/discover
- GET /discoveries/{id}
- GET /applications/{id}/appgraph
- GET /applications/{id}/objects

### Requirements

- GET /projects/{id}/requirements
- POST /projects/{id}/requirements
- PATCH /requirements/{id}
- POST /requirements/analyze
- GET /requirements/{id}/coverage
- GET /requirements/{id}/traceability

### Tests

- GET /projects/{id}/tests
- POST /projects/{id}/tests
- PATCH /tests/{id}
- POST /tests/generate
- GET /test-generation/{id}
- GET /tests/{id}/test-intent
- PUT /tests/{id}/test-intent
- POST /tests/{id}/automation/generate

### Execution

- POST /tests/execute

- GET /executions/{id}
- GET /executions/{id}/steps
- GET /executions/{id}/artifacts
- POST /executions/{id}/cancel
- POST /executions/{id}/retry

### Failures/Healing

- POST /failures/analyze
- GET /failures/{id}
- POST /tests/heal
- GET /healing/{id}
- POST /healing/{id}/approve
- POST /healing/{id}/apply
- POST /healing/{id}/rollback

### Regression/Risk

- POST /regression/analyze
- GET /regression/{id}
- POST /regression/{id}/execute
- GET /projects/{id}/release-readiness

### Defects

- GET /projects/{id}/defects
- POST /defects/create
- GET /defects/{id}
- PATCH /defects/{id}
- POST /defects/{id}/link
- POST /defects/{id}/sync

### Evidence

- GET /evidence/{id}
- GET /evidence/{id}/graph
- GET /evidence/{id}/artifacts
- POST /evidence/packages

### AI/Copilot

- POST /copilot/query
- GET /ai/runs/{id}
- GET /ai/runs/{id}/tools
- POST /ai/actions/{id}/approve
- POST /ai/actions/{id}/reject

### Reports

- GET /projects/{id}/dashboard
- GET /projects/{id}/coverage
- GET /projects/{id}/flaky-report
- POST /reports
- GET /reports/{id}
- GET /reports/{id}/download

### Integrations

- GET /projects/{id}/integrations
- POST /projects/{id}/integrations
- POST /integrations/{id}/test
- POST /integrations/{id}/sync
- POST /projects/{id}/webhooks

### Platform

- GET /search
- POST /files
- GET /files/{id}/download
- GET /projects/{id}/audit-logs
- GET /projects/{id}/activity

## 29.2 Core request/response examples

```
POST /api/v1/tests/execute
{
  "project_id":"uuid",
  "test_ids":["uuid"],
  "environment_id":"uuid",
  "adapter":"playwright",
  "browser_matrix":[{"browser":"chromium","version":"stable"}],
  "priority":"P1",
  "retry_policy":{"max_attempts":1},
  "evidence_policy":"diagnostic",
  "healing_policy_id":"uuid"
}

202
{
  "execution_id":"uuid",
  "workflow_id":"uuid",
  "status":"QUEUED",
  "queued_at":"2026-08-23T00:00:00Z",
  "correlation_id":"uuid"
}

POST /api/v1/defects/create
{
  "failure_id":"uuid",
  "title":"Checkout fails when duplicate coupon is applied",
  "severity":"HIGH",
  "priority":"P1",
  "environment_id":"uuid",
  "build":"2026.08.23.17",
  "steps":[...],
  "expected":"Coupon is rejected with validation message",
  "actual":"500 response",
  "evidence_refs":["uuid","uuid"],
  "external_sync":{"system":"jira","project_key":"QA"}
}
```

## 30. Event-Driven Execution & WebSocket

| Event | Minimum envelope |
|---|---|
| RequirementCreated | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| RequirementUpdated | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |

| | |
|---|---|
| DiscoveryStarted | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| DiscoveryCompleted | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestGenerated | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestApproved | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestQueued | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestStarted | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestStepStarted | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestStepPassed | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestStepFailed | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| EvidenceCaptured | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| FailureDetected | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| FailureAnalyzed | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| HealingAttempted | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| HealingSucceeded | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| HealingRejected | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| RegressionAnalyzed | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| DefectCreated | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| DefectSynced | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| ReleaseRiskCalculated | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| QualityGateEvaluated | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |
| TestCompleted | event_id, tenant_id, project_id, entity_id, workflow_id, actor_type, timestamp, correlation_id, payload |

## 30.1 Execution WebSocket

```
WS /api/v1/ws/executions/{execution_id}
```

Stream execution status, step state, timing, browser/worker status, artifact availability, FailureIQ and healing events. Do not stream secrets or hidden model chain-of-thought.

# 31. Security, Privacy & AI Safety

- SSO: SAML/OIDC/OAuth/LDAP
- MFA
- RBAC and tenant isolation
- Encryption in transit/at rest
- Secret manager/Vault/KMS
- API key rotation
- PII detection/masking
- Data retention/deletion
- Audit trail
- AI model allowlist

- Prompt registry
- AI action permissions
- Confidence thresholds
- Usage quotas
- Data residency
- IP allowlisting
- Backup/restore
- Disaster recovery
- Prompt-injection defense

## 31.1 AI safety rules

- Treat requirements, webpages, logs, API responses and uploaded documents as untrusted content.
- Retrieved text cannot override platform policy/system instructions.
- Tool calls must use allowlisted schemas.
- Secrets are represented by references, not raw values.
- High-impact actions require policy + permission + confidence threshold.
- Critical test changes must be reversible.
- AI-generated business rules require source traceability or human confirmation.

# 32. Development Tools, Technologies & Engineering Techniques

| Area | Recommended tools/techniques |
|---|---|
| Frontend | React, TypeScript, Next.js; component library; state/query management; WebSocket client; charting; form validation |
| Backend | Python, FastAPI; Pydantic; SQLAlchemy; async workers; OpenAPI |
| AI orchestration | LangGraph, Semantic Kernel or controlled workflow engine; tool gateway; structured outputs |
| LLM | Provider-agnostic AI Gateway; model routing; embeddings; evaluation |
| Automation | Playwright; Selenium; browser drivers; later Appium |
| API | HTTP client layer; REST/GraphQL/SOAP adapters |
| Database | PostgreSQL; pgvector; SQLAlchemy/Alembic |
| Cache/queue | Redis; RabbitMQ initially where event semantics require; Kafka at scale |
| Artifacts | S3-compatible object storage |
| Search | OpenSearch/Elasticsearch where required |
| Containers | Docker |
| Orchestration | Kubernetes; Helm |
| Ingress/API | Nginx/HAProxy/API gateway |
| CI/CD | GitHub Actions/Jenkins/GitLab CI/Azure DevOps |
| IaC | Terraform |
| Monitoring | Prometheus + Grafana or equivalent |
| Logging | Structured JSON logs + OpenSearch/ELK |
| Tracing | OpenTelemetry + Jaeger/Tempo |
| Secrets | Vault/KMS/managed cloud secrets |
| Security testing | OWASP ZAP, Semgrep, SonarQube |
| Accessibility | axe-core |
| Performance | k6/JMeter |
| Mobile | Appium |
| Testing | pytest, Playwright test suites, API contract tests, integration/e2e, AI evaluation harness |
| Quality | Ruff/Black/isort/mypy or equivalent; ESLint/Prettier; pre-commit |
| Documentation | OpenAPI, ADRs, architecture diagrams, module specs |

## 32.1 Engineering techniques

- API-first development
- Adapter pattern

- Domain-driven module boundaries
- Event-driven execution
- Asynchronous job processing
- Idempotency
- Schema-first contracts
- Feature flags
- Contract testing
- Test pyramid
- Infrastructure as code
- Container isolation
- Blue/green or canary deployment
- Observability-first development
- AI evaluation-driven development
- Prompt versioning
- Human-in-the-loop for high-impact autonomy

## 33. Recommended Repository Structure

```
oneclickqa/
  apps/
    web/
    api/
  services/
    identity/
    tenant/
    project_management/
    requirement_intelligence/
    application_intelligence/
    test_intelligence/
    ai_orchestration/
    execution/
    evidence/
    defect/
    reporting/
    integrations/
    governance/
  workers/
    playwright/
    selenium/
    api/
    database/
    mobile/
    performance/
    security/
  packages/
    contracts/
    testintent/
    policy_engine/
    ai_gateway/
    evidence_graph/
    event_bus/
    observability/
  infra/
    docker/
    kubernetes/
    helm/
    terraform/
  tests/
    unit/
    integration/
    contract/
    api/
```

```
    e2e/
    execution/
    ai_evaluation/
    security/
  docs/
    api/
    adr/
    architecture/
    runbooks/
```

# 34. Internal Processing Standards

## 34.1 Synchronous vs asynchronous

| Operation | Mode |
|---|---|
| Create/update metadata | Synchronous |
| Search/filter | Synchronous |
| Start discovery | Async |
| Generate tests | Async |
| Generate test data | Async |
| Execute tests | Async |
| Failure analysis | Async where evidence volume is high |
| Healing | Async |
| Regression analysis | Async |
| Report generation | Async |
| External synchronization | Async |
| File upload | Sync metadata + async processing |

## 34.2 Standard job lifecycle

```
REQUESTED → VALIDATING → QUEUED → RUNNING → WAITING_APPROVAL → RETRYING → SUCCEEDED / FAILED / CANCELLED
```

## 34.3 Standard failure recovery

- Retry transient infrastructure failures.
- Do not retry deterministic assertion failures unless retry policy explicitly says so.
- Reassign jobs from dead workers.
- Persist checkpoints where possible.
- Never duplicate external defect creation; use idempotency/external ID.
- Preserve all partial evidence even when a job fails.

# 35. Complete Acceptance Criteria

- User can create a tenant/project and assign users with RBAC.
- User can register an application and environment using secret references.
- Discovery produces a versioned AppGraph with pages and objects.
- User can upload/paste a requirement/user story and receive structured requirements and coverage analysis.
- User can review and approve AI-generated test scenarios.
- Approved scenario becomes versioned TestIntent.
- TestIntent can be executed through Playwright and Selenium.

- Chrome, Firefox and Edge are supported in initial web MVP.
- Execution supports parallel jobs and real-time status.
- Execution captures configured evidence.
- FailureIQ classifies failures with evidence references.
- AutoHeal produces ranked candidates and confidence.
- AutoHeal respects >95/80–95/<80 policy boundaries.
- Any applied healing is re-executed and validated.
- RiskRun creates impacted test set and explainable risk.
- EvidenceGraph provides requirement-to-defect traceability.
- Users can raise defects from all relevant failure/testing contexts.
- Defect package contains required reproduction, expected/actual, environment, build and evidence information.
- External defect synchronization is idempotent.
- CI/CD can trigger execution and receive quality-gate status.
- Dashboard shows execution, coverage, defect, healing and release risk metrics.
- All AI actions are permission/policy controlled and auditable.
- Secrets never appear in logs/prompts.
- Async jobs survive worker failure without corrupting system state.
- API documentation and contract tests exist for every public endpoint.
- All critical paths have unit, integration and end-to-end tests.

# 36. Validation & Test Strategy for OneClickQA Itself

- Unit testing: domain logic, parsers, scoring, policy.
- API tests: schema, auth, tenant isolation, validation, idempotency.
- Contract tests: service/event/API compatibility.
- Integration tests: PostgreSQL/Redis/object storage/AI gateway.
- Automation tests: Playwright/Selenium adapters on reference applications.
- End-to-end tests: requirement→test→execution→failure→healing→defect→report.
- AI evaluation: fixed datasets and golden answers for every agent.
- Security tests: SAST, dependency scan, DAST, secret scan, prompt injection tests.
- Performance tests: API latency, queue throughput, worker scaling.
- Resilience tests: worker crash, queue retry, storage outage, model timeout.
- Browser matrix tests.
- Migration/backward compatibility tests.

# 37. Monitoring, Logging, Alerting & Operations

## 37.1 Metrics

- API latency/error rate
- Queue depth
- Worker utilization
- Execution duration
- Pass/fail/flaky
- Healing attempt/success
- AI latency/token/cost
- Failure classification confidence
- Defect creation rate

- Integration sync failures
- Artifact upload failures
- Storage growth
- Database latency/connections

## 37.2 Logs

- Application logs
- API logs
- Worker/execution logs
- Security logs
- AI interaction metadata
- Audit logs
- Integration logs
- Scheduler logs

## 37.3 Tracing

Trace from API request → workflow → agent/tool → queue → worker → adapter → evidence → analysis → defect/report using correlation_id, workflow_id and execution_id.

## 37.4 Alerts

- Worker exhaustion
- Queue backlog
- Failed scheduled jobs
- Storage failure
- Model/API failure
- Integration authentication failure
- Database connection saturation
- High error rate
- High flaky rate
- Healing regression detected

# 38. Deployment & Environment Strategy

| Environment | Requirements |
|---|---|
| Local | Docker Compose; local PostgreSQL/Redis/object store; mock integrations |
| Development | Shared dev cluster; disposable workers; development AI models |
| QA/Test | Stable test integrations; seeded datasets; automated regression |
| Staging/UAT | Production-like topology; external integration validation; release candidate |
| Production | Kubernetes/managed infrastructure; autoscaling; hardened secrets; backup/DR |

## 38.1 Deployment pipeline

115. Commit

116. Lint/static analysis
117. Unit tests
118. Build containers
119. Dependency/security scan
120. Integration tests
121. Contract tests
122. Deploy to dev
123. Smoke tests
124. Promote to QA
125. Regression
126. Deploy staging
127. UAT/release approval
128. Production deployment
129. Post-deployment health check
130. Monitor/rollback

## 39. Development Work Packages / Team Ownership

| Team | Primary ownership |
|---|---|
| Frontend | Web shell, dashboards, module screens, forms, live execution, evidence viewer, defect UX, copilot |
| Backend | API gateway, domain services, persistence, auth, workflows, reporting |
| AI/ML | Agents, prompts, retrieval, AI gateway, evaluations, scoring, confidence |
| Automation | Playwright/Selenium/API adapters, object intelligence, execution framework |
| QA Domain | Test types, acceptance rules, FailureIQ taxonomy, quality gates, UAT |
| DevOps/SRE | Docker, Kubernetes, queues, observability, CI/CD, secrets, DR |
| Security | IAM, tenant isolation, secret controls, SAST/DAST, AI safety |
| Data | PostgreSQL/pgvector schema, migrations, indexes, evidence storage, knowledge graph |

The source blueprint recommends a focused cross-functional MVP team including Product/QA Architect, AI/ML Engineers, Backend Engineers, Automation Engineers, Frontend, DevOps/SRE, QA/Domain and UX/Product capacity. filecite turn8file9

## 40. Scope Strategy

| Stage | Capabilities |
|---|---|
| MVP | Web UI, Chrome/Firefox/Edge, Playwright/Selenium, requirement-to-test, TestIntent, AI test data, one-click execution, evidence, FailureIQ basic, confidence-based AutoHeal, core reporting, basic CI/CD |
| Expansion 1 | REST/GraphQL, database validation, advanced regression/risk, mobile |
| Expansion 2 | Performance, accessibility, security integrations, production telemetry, autonomous regression, advanced governance |
| Full Platform | All feature-matrix capabilities, enterprise integrations, predictive quality, service virtualization, exploratory/UAT, comprehensive reporting |

The source roadmap explicitly places API/GraphQL/database/mobile in the next expansion and performance/accessibility/security/production telemetry/autonomous regression/advanced governance later. filecite turn8file9

## 41. Requirements Traceability Matrix

| ID | Business | AI | Service | Data | API | Output |
|---|---|---|---|---|---|---|

| | requirement | | | | | |
|---|---|---|---|---|---|---|
| REQ | Requirement/User Story | Requirement Intelligence | Requirement service | Requirement tables | Requirement APIs | Requirement coverage |
| APP | Application understanding | Application Discovery | Application service | AppGraph tables | Discovery APIs | AppGraph/evidence |
| TEST | Test generation | Test Design Agent | Test Intelligence | Tests/TestIntent | Generate APIs | Test coverage |
| DATA | Test data | Test Data Agent | Data service | Datasets | Dataset APIs | Data evidence |
| EXEC | Execution | Execution Intelligence | Execution service | Executions/steps | Execute APIs | Execution report |
| FAIL | Failure diagnosis | FailureIQ | Failure service | Failures | Analyze APIs | Failure evidence |
| HEAL | Self-healing | AutoHeal | Healing service | Healing/selector memory | Heal APIs | Healing report |
| RISK | Risk regression | RiskRun | Risk service | Risk/impact | Regression APIs | Release report |
| EVID | Traceability | Evidence Intelligence | Evidence service | Evidence graph | Evidence APIs | Evidence explorer |
| DEF | Defects | AI Defect Intelligence | Defect service | Defects | Defect APIs | Defect report |
| REL | Release decision | Release Intelligence | Reporting service | Quality gates/release | Release APIs | Release dashboard |

# 42. Definition of Done — Mandatory for Every Module

- Functional requirement implemented.
- UI states complete.
- API documented in OpenAPI.
- Request validation implemented.
- Authorization implemented and tested.
- Database migration/indexes added.
- Events emitted where required.
- Audit event added where required.
- Observability metrics/logs/traces added.
- Unit tests added.
- Integration/contract tests added.
- E2E path added for critical workflows.
- AI prompt/model versioned and evaluation updated where AI is involved.
- Security review completed.
- Evidence/traceability verified.
- Rollback/feature flag defined for risky autonomous behavior.
- Documentation updated.
- No raw secret in database/log/prompt.

# 43. Final End-to-End System Workflow

The final platform must behave as a closed-loop autonomous quality system rather than a collection of disconnected testing utilities.

131. INPUT: application URL, API specification, requirement, user story, existing test or natural-language QA objective.
132. UNDERSTAND: Requirement Intelligence + Application Discovery + AppGraph.
133. DESIGN: Test Intelligence + TestIntent + Test Data Intelligence.
134. AUTOMATE: Playwright/Selenium/API and future adapters.
135. EXECUTE: distributed execution fabric.
136. OBSERVE: screenshot, video, DOM, network, console, API/DB evidence and telemetry.
137. DIAGNOSE: FailureIQ + historical knowledge.
138. HEAL: AutoHeal for eligible automation failures.
139. PROVE: EvidenceGraph creates requirement-to-execution-to-defect traceability.

140. REPORT: dashboards, defect reports, coverage and release readiness.
141. DECIDE: RiskRun + Release Intelligence + quality gates.
142. LEARN: QA Knowledge Graph updates from validated outcomes.

This sequence directly reflects the supplied blueprint's ultimate workflow and strategic positioning: understand → generate → execute → diagnose → heal → prove → decide. filecite turn7file10

# Appendix A — Explicit Source Coverage

The source blueprint is organized into product vision, seven-layer architecture, experience, copilot, orchestration, agent suite, requirement intelligence, AppGraph, object intelligence, TestIntent, test data, automation/execution, execution intelligence, FailureIQ, AutoHeal, RiskRun, EvidenceGraph, defect intelligence, QA Knowledge Graph, flaky intelligence, governance, AI gateway, prompt registry, data architecture, service architecture, API-first, events, CI/CD, release intelligence, reporting, technology stack, MVP and delivery strategy. filecite turn8file0

The feature matrix explicitly covers ten testing-capability groups and additional intelligent QA, test management, test data, execution, CI/CD, reporting, collaboration, governance and platform services. filecite turn9file15

The supplied architecture reference visually decomposes the platform into access channels, five core product capability groups, platform services, data/AI, infrastructure, then API gateway, microservices, common services, test execution framework, adapters, isolated workers, artifacts, result aggregation and detailed data stores. This structure has been incorporated into Sections 3 and 32 of this document.

# Appendix B — Next Planning Artifact

This document intentionally does not replace the requested future 7-week roadmap. The roadmap should be generated only after this baseline is accepted. It should map every day to the modules, APIs, UI screens, AI agents, database work, automation adapters, infrastructure, integration, QA validation and acceptance criteria defined here.

The result should be a day-wise engineering plan with dependencies, owners, deliverables, code-level work packages, review gates, test activities and integration milestones.